

Lecture 11: Job Scheduling with Time Budget

Date: August 12, 2026
Instructor: Aniket Basu Roy
Scribe: Nitish Katteboyina and Sumedh Shete

Overview. In this lecture we study a scheduling problem in which a set of jobs is given, each requiring an execution time. A machine is available for only a limited Time Budget (W). Our aim is to select a subset of jobs whose total execution time does not exceed W while maximizing the utilized time. We formalize the problem, start by building a brute force approach, design an optimal algorithm for the problem, and analyze its complexity.

1 Problem Definition

Motivation. Assume there is a job scheduler which needs to assign jobs to a processor for processing within a limited time budget. Since the available time is limited, only a particular number of jobs can be executed. Here, CPU utilization depends on choosing the right subset of jobs, as no one wants to waste processor time. Using an algorithm, we find a subset that maximizes CPU utilization. Therefore, we need to find a subset whose total execution time is maximum while remaining less than or equal to the given time budget.

Definition 1 (Instance). An instance consists of a set of jobs

$$J = \{J_1, J_2, \dots, J_n\}$$

where each job J_i has an execution time (burst time) b_i , together with a time budget W .

Definition 2 (Feasible Schedule). A subset of jobs $S \subseteq J$ is called a feasible schedule if

$$\sum_{J_i \in S} b_i \leq W.$$

Definition 3 (Optimal Schedule). A feasible schedule S is optimal if the total execution time of the selected jobs is maximum among all feasible schedules.

2 Worked Example

Consider the following burst times: $\{5, 8, 11, 13, 17, 6, 9\}$ and Time Budget $W = 40$.

One possible subset is $\{17, 13, 9\}$ whose total execution time is 39. Another possible subset is $\{17, 11, 6, 5\}$ whose total execution time is also 39.

A subset such as $\{17, 11, 8, 5\}$ has a total execution time of 41, which is greater than the time budget. Hence, it cannot be selected.

In this example, no subset has a total execution time exactly equal to 40. The best utilization we can achieve is 39. Therefore, our goal is to find a subset whose total execution time is as large as possible without exceeding the given time budget.

3 Approach I: Brute Force Recursion

3.1 Key Idea

For every job, we try including it in the current subset and continue exploring the remaining jobs. Starting from a given index, we try every remaining job as the next job in the schedule. The maximum feasible execution time found during this process is returned.

3.2 Recursive Formulation

Let $\text{recursion}(\text{index}, \text{sum}, W, A)$ represent the maximum utilization obtainable starting from position **index**, where **sum** is the current execution time.

The recursion explores all valid subsets and returns the maximum feasible sum.

Algorithm 1: Brute Force Recursive Solution

```

1 Function recursion(index, sum, W, A)
2   if sum > W then
3     return 0
4   ans ← sum
5   for i ← index to A.length − 1 do
6     if sum + A[i] ≤ W then
7       ans ← max(ans, recursion(i + 1, sum + A[i], W, A))
8   return ans

```

3.3 Complexity Analysis

Every job can either be selected or not selected. Therefore, all possible subsets may be explored.

$$\text{Time Complexity} = O(2^n)$$

$$\text{Space Complexity} = O(n)$$

where n is the number of jobs.

4 Approach II: Dynamic Programming

4.1 Motivation for DP

In the recursive solution, the same subproblems are solved multiple times. Dynamic Programming stores previously computed results and avoids repeated work.

4.2 DP State Definition

Let $dp[i][w]$ denote the maximum execution time that can be achieved using the first i jobs with a time budget w .

4.3 Recurrence Relation

If the current job cannot fit into the remaining budget,

$$dp[i][w] = dp[i - 1][w].$$

Otherwise,

$$dp[i][w] = \max \left(dp[i - 1][w], A[i] + dp[i - 1][w - A[i]] \right).$$

The first option skips the current job, while the second option selects it.

4.4 Algorithm

Algorithm 2: Dynamic Programming Solution

Input: Burst time array A and budget W
Output: Maximum feasible utilization

```

1 Create table  $dp[n+1][W+1]$  initialized with 0;
2 for  $i \leftarrow 1$  to  $n$  do
3   “ for  $w \leftarrow 0$  to  $W$  do
4     if  $A[i-1] \leq w$  then
5        $dp[i][w] \leftarrow \max(dp[i-1][w], A[i-1] + dp[i-1][w - A[i-1]])$ 
6     else
7        $dp[i][w] \leftarrow dp[i-1][w]$ 
8   “
9 return  $dp[n][W]$ ;
```

4.5 Complexity Analysis

The DP table contains

$$(n+1)(W+1)$$

entries and each entry is computed in constant time.

$$\text{Time Complexity} = O(nW)$$

$$\text{Space Complexity} = O(nW)$$

5 Comparison of Approaches

Approach	Time Complexity	Space Complexity
Brute Force Recursion	$O(2^n)$	$O(n)$
Dynamic Programming	$O(nW)$	$O(nW)$

The dynamic programming approach is significantly faster than the brute force approach because it avoids solving the same subproblems multiple times.

6 Conclusion

In this lecture, we studied a job scheduling problem with a limited time budget. We first developed a brute force recursive solution that explores all possible subsets of jobs. Although correct, its running time grows exponentially with the number of jobs. We then improved the solution using dynamic programming, which stores intermediate results and reduces the running time to $O(nW)$. This allows us to efficiently find the subset of jobs that maximizes CPU utilization without exceeding the given time budget.